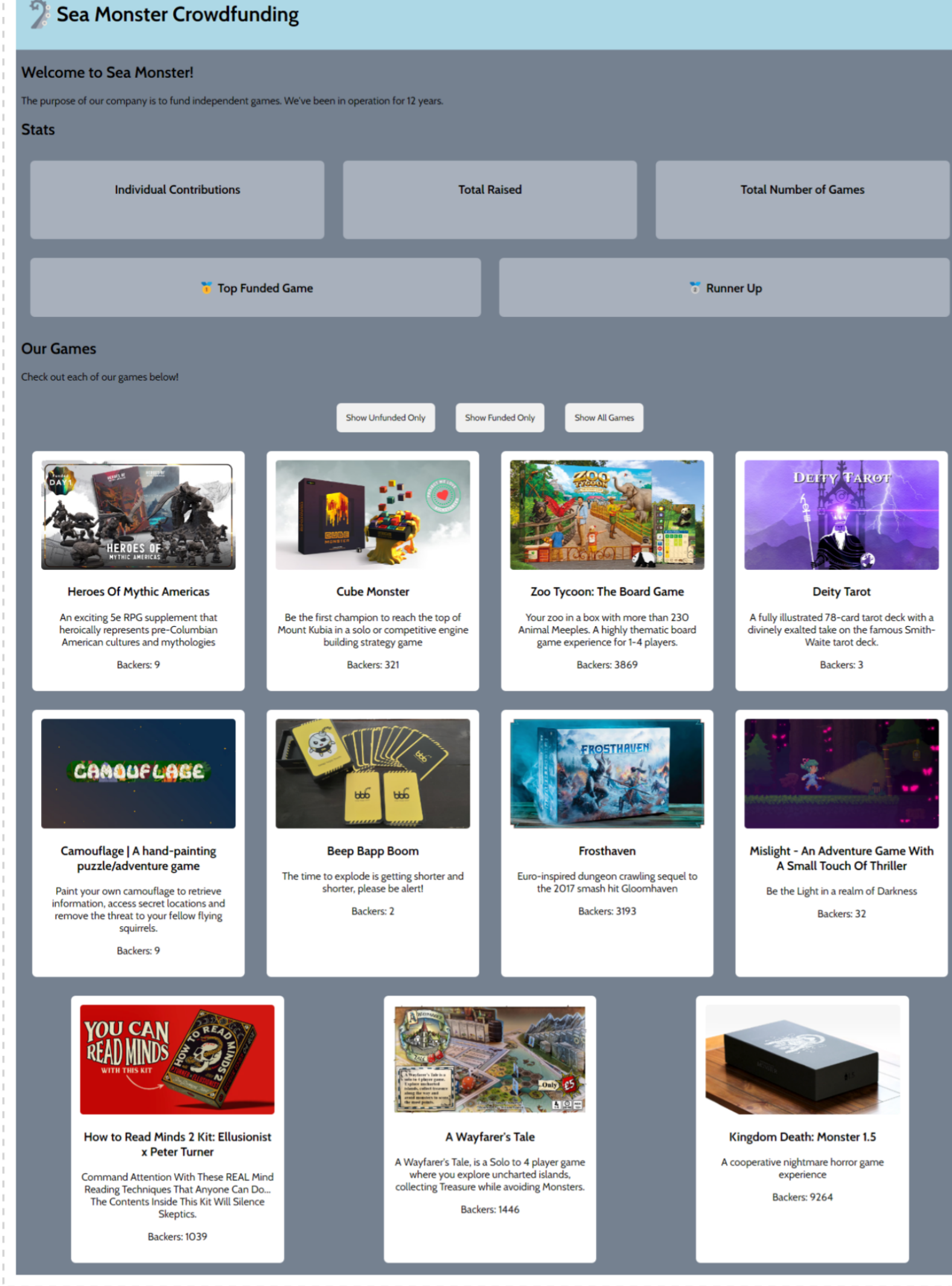


Challenge 4 of 7: Create the summary statistics at the top of the page

✓ Checkpoint

Awesome work solving Challenge 3 and welcome to Challenge 4! In this challenge, you'll fill out the Stats section at the top of the page to display some neat data about the different games funded by Sea Monster.

At this point, your website should look something like the screenshot below. Compare your work with the exemplar and see if there are any changes you might need to make!



🖌 Background

In this section, we'll be using arrow functions several times. Recall that functions can be passed as arguments to other functions.

For one of our Stats cards, we need to find the total amount of money raised across all games. Think about how you would solve this problem using the tools you already have. You would probably consider using a loop to sum up contributions. That's a lot of repetitive code to write and I don't know about you, but I could go the rest of my life without writing `for (let i = 0; i < list.length; i++)` again.

Fortunately, JavaScript has a great solution for writing this type of algorithm in the form of `reduce`. `Reduce` collapses a list into a single value. In this case, we take a list of games and collapse that into a single number: the total amount raised.

`reduce` takes two arguments: a callback and the initial value.

- **Callback:** a two-argument function that runs once for each element of the list, telling JavaScript what to do with that element to produce the final value. The two arguments are the accumulator variable, which stores the values of all previous calls together, and the list element itself.
- **Initial value:** the starting value for the accumulator variable.

Let's look at an example.

Imagine we work at Spotify and we're working on Spotify Wrapped. We have a list of a user's songs called `songs` and want to display that user's total number of listens across all songs. Here's how we could use `reduce` to do that:

```
const totalListens = songs.reduce( (acc, song) => {  
  return acc + song.listens;  
}, 0);
```

That's not a lot of code! Let's break this function down:

- `totalListens` stores the final value of `reduce`: the number of listens across all songs
- `reduce` takes two arguments: the function used to sum up the listens and the starting point for the calculation: 0
- The callback within `reduce` is an anonymous arrow function (an arrow function without a name)
 - `acc` stores all the listens summed up to that point in the list of songs
 - `song` is effectively the same as `songs[i]` in a traditional `for` loop: it's the current song in the list

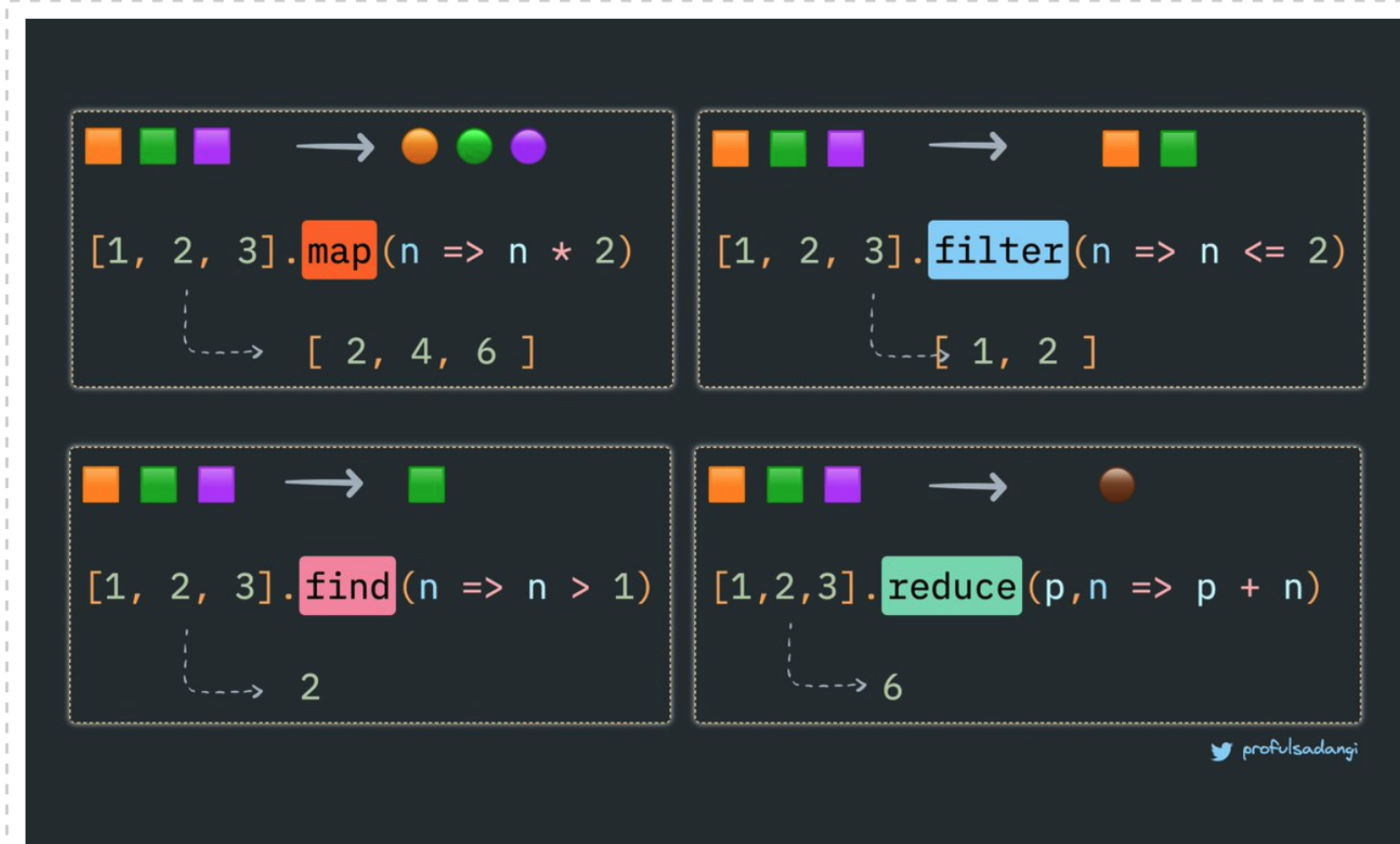
As a beginner, one weird part of understanding `reduce` is the accumulator. If you're trying to wrap your head around it, consider this metaphor:

- Imagine a charity worker collecting donations from a group of neighbors. They visit each person one by one and ask them to drop some change into a bucket. In this example, the bucket acts as the accumulator: it holds all the change collected up to that point. At first, there are no donations in the bucket because no one has been visited yet. But the amount in the bucket grows as the charity worker visits each person. At the end of the trip, the bucket contains the accumulated donations from all the neighbors: this is the final value returned to the charity by the worker.

If you'd like to read more about `reduce` before tackling these challenges, check out these resources:

- [Array.prototype.reduce docs](#)
- [JavaScript Array reduce & reduceRight: Reducing an Array Into a Value](#)

We'll be looking at a few more `Array.prototype` functions in the upcoming challenges. Here's a helpful image that uses color to explain how some of these functions work (including `reduce`) (image credit: Proful Sadangi):



Finally, before we move on, let's check out one more tool: the `.toLocaleString()` function. This function takes one argument, a language code, and returns a number formatted with commas and decimal points as it would be represented in that language. Here's an example:

```
let num = 444333111.222  
console.log(num.toLocaleString('en-US')); // US English code  
// => 444,333,111.222
```

Let's

get started on the challenge!

🚀 Challenge Steps

Navigate to the Challenge 4 section of `index.js` to begin.

➡➡➡ Step 1

Use `reduce` on the `GAMES_JSON` array to calculate the total number of individual contributions made to games. Change the value of the `contributionsCard` to display the result.

- Note: "Individual contributions" means the number of backers to each game.

🔑 **Secret Key component 1:** What value is now displayed under the Individual Contributions heading?

Please only include the number (for example, if `25,674` is displayed, you would include `25674` in your Secret Key).

➡➡➡ Step 2

Calculate the total amount of money pledged across all games. Display this value in the `raisedCard` with a dollar sign.

🔑 **Secret Key component 2:** What value is now displayed under the Total Raised heading? Please only include the number (for example, if `$3,876` is displayed, you would include `3876` in your Secret Key).

➡➡➡ Step 3

Display the total number of games in the `gamesCard`.

🔑 **Secret Key component 3:** Identify the error in this code segment:

```
let animals = ["giraffe", "horse", "narwhal", "chinchilla"];  
let firstLetters = animals.reduce( (sum, animal) => {  
  return animal.charAt(0);  
}, "");
```

1. The first argument to the arrow function should be called `acc` (keyword: FOIL)
2. `reduce` can only be called on an array of objects (keyword: DOODLE)
3. The arrow function does not add `sum` and `animal.charAt(0)`, so it will return only `"c"` (keyword: BRAIN)
4. The second argument to `reduce` should be the number 0, not the empty string (keyword: TUSK)

🚀 Moving on to Challenge 5

Create the 🔑 Secret Key from its components with no spaces in between:

- Component 1: What value is now displayed under the Individual Contributions heading?
- Component 2: What value is now displayed under the Total Raised heading?
- Component 3: Identify the error in this code segment.

Hint: your 🔑 **Secret Key** should be 16 characters long.

Reminders about 🔑 **Secret Keys**:

- Keys for the Offline PDF version of prework **are case sensitive!**
 - (For example, "SEA_MONSTER" is NOT the same as "sea_monster")
- Secret keys can be any combination of letters, numbers, and symbols.

Use your 🔑 **Secret Key** to unlock the next PDF file. If your 🔑 **Secret Key** is correct, you will move onto the next step!